

Can we trust this agent? A support agent for @SpotifyCares

What this is. An AI support agent for Spotify's public Twitter/X support handle. It classifies an incoming customer message into one of 10 intents induced from the data, drafts a reply grounded in how Spotify actually handled similar messages, and decides auto-send vs escalate-to-human with a stated reason. It runs on free open-weights models locally — `qwen2.5:7b-instruct` for the agent, `llama3.1:8b` as judge, no API keys.

The build was the easy half. This report is mostly about the other half.

The headline, stated the way I would defend it in a review:

	agent (S3)	copy-paste baseline (B1)
Trustworthy Automation Rate — send-ready and correctly auto-routed	10.5%	21.0%
Unsafe auto-handle rate — judge-independent, and the one that gates deployment	6.5%	14.0%
Escalation recall	86.7%	71.4%
Intent macro-F1	0.67	0.295

My agent loses to a baseline that copies a retrieved reply verbatim, on my own headline metric. It wins on everything the judge does not touch. §3.3 and §5.3 argue — with evidence, not indignation — that the judge is wrong about B1 and that the ranking is an artefact of a rubric I wrote. But the composite as computed does not favour my system, and I am not going to bury that in an appendix.

§5 is the full argument for discounting every number above, including two places where auditing my own work destroyed a result I liked.

1. Problem framing: what "good" means, and what I did not build

Spotify's public support handle answered 41,383 customer turns across 28,280 threads here; the median thread is 2 turns and the average reply 127.8 characters. I read ~450 threads before writing code. Three facts shaped everything after.

It is a triage desk, not a resolution desk. The commonest outbound action is a *next step*, not an answer — ask for device/OS/version, or ask for the account email in DM (31.8% of replies request one). So "a good reply" here usually means *the correct next step*; evaluating on "did it solve the problem" would measure something Spotify's own agents do not do in public.

The automatable and the risky separate cleanly — and not by topic. Anything needing a look at *this customer's* account, plan or money goes to DM every time. Everything else — a missing album, a feature request, a device bug, a thank-you — is answered publicly with a stable, near-templated response. That is the automation boundary, and it is why routing keys off *risk signals* rather than intent alone.

The tail carries the damage. In the held-out pool: money 9.3%, security 2.4%, churn-or-human-request 1.8%, non-English 0.5%, legal/fraud 0.3%. A uniform sample of 200 would hold ~5 security cases, ~1 non-English message and probably zero legal ones — it could not measure the thing most worth measuring. Hence a stratified golden set.

So "good" is three things, weighted unevenly

- Never auto-send something that needed a human.** The only hard constraint. A missed escalation on a fraud claim is real harm; an unnecessary escalation costs one agent-minute. Metric: **unsafe auto-handle rate**. I care about it more than anything else here.
- Automate enough volume to be worth deploying.** An agent that escalates everything is perfectly safe and perfectly useless. Metric: **auto coverage**, meaningless without (1).
- Draft replies a human would send unedited.** Not "reads nicely" — *ships*. Metric: **send-ready rate**, where any one rubric dimension below 4/5 fails the whole reply.

Combined into the headline, **Trustworthy Automation Rate (TAR)**: the share of all messages where the agent chose auto, gold agrees that was safe, *and* the draft was send-ready — a composite, because every single-axis metric here has a trivial winner. §5 is about how even TAR misleads.

What I chose not to build

Not a conversation owner — one reply to one message, no cross-turn state, no DM flow. **Not Banking77** — a clean 77-class number that would tell a Spotify support lead nothing; I looked and dropped it. **No fine-tuning** — with 200 labels, prompting plus retrieval is the honest ceiling. **No deflection/CSAT** — I cannot observe what a draft would do to a real customer, and §5.10 says so. **No dedicated safety model** — self-harm routes to a human via one risk signal, which is not good enough for production.

2. The system and the harness

2.1 The agent: LLM for perception, code for policy

① **Perception (LLM).** One call returns the intent (10 codes, `data/taxonomy.yaml`), a confidence band, and 10 boolean risk signals — `account_specific`, `money_involved`, `security_or_privacy`, `legal_or_regulatory`, `human_requested`, `churn_threat`, `safety_or_wellbeing`, `non_english`, `abusive_or_public_escalation`, `vague_no_detail`. The prompt carries the same codebook the annotator used.

② **Policy (~25 lines of Python)**. `auto` requires all of: intent auto-eligible, no hard risk signal, confidence not `low`. Otherwise escalate, with the failing condition returned as the reason ("risk signal `money_involved`"). Auditable, unit tested, and the risk appetite is a YAML edit. It also makes failure attribution possible — §4 F2 exists only because of it. Ablation S4 measures what happens when the LLM decides instead.

③ **Generation (LLM)**. A second call drafts the reply given the four most similar past messages Spotify actually answered, real replies attached, under the rule that every commitment must appear in that evidence. The grounding store is TF-IDF over word 1–2-grams plus `char_wb` 3–5-grams, fitted only on the 70% time-ordered history split; a unit can never retrieve its own conversation. Median best-neighbour similarity on the golden set is 0.337 and only 1.0% of units have a near-verbatim twin — analogy, not lookup.

2.2 The golden set (200 hand-labelled units)

Held-out pool de-duplicated to 11,644 answered turns, partitioned into 7 keyword strata (first match wins, so partition and weights are exact), with quotas per stratum: 120 `general`, 18 `money`, 16 `security`, 16 `churn_human`, 10 each `safety_legal` / `non_english` / `vague_short`. Every unit carries `weight = stratum population / sampled`, which is what makes the reweighted column possible (frame: `data/golden/frame_report.json`; protocol: `data/golden/LABELLING.md`).

One annotator (me), seeing only the customer message and up to four earlier turns. **Spotify's actual reply was withheld by the tooling** — otherwise my "should this escalate?" label would have collapsed into "did Spotify ask for a DM?", and I would be evaluating the agent against a proxy for itself. Per unit: gold intent, an acceptable alternative intent, gold route, the single escalation driver, a `hard` flag, a note.

Two signal definitions were tightened after ~100 units showed they were too loose (`money_involved` no longer fires on general price questions; `churn_threat` no longer on routine cancellation admin or jokes) and the affected units re-labelled — indices in `LABELLING.md`.

46 of 200 units are flagged `hard` — 23%. That is the most honest statistic here: a quarter of real support messages have no obvious right answer, so a model scoring 100% would mean my labels were wrong.

2.3 The judge

Pointwise, blind, four dimensions on 1–5 against written anchors (`prompts/judge.txt`): **groundedness**, **resolution**, **tone**, **safety**, plus extraction of the phrases it considers unsupported (which is what makes §4 F4 possible). Candidates from all systems are pooled and shuffled; the prompt never names the system, including for Spotify's own replies.

`send_ready` is **not** the judge's boolean. It is `min(all four) >= 4`, computed in code — a policy knob I can defend rather than a mood. The judge's own `send_as_is` is reported alongside and is systematically looser.

The judge is `llama3.1:8b`, a different family from the `qwen2.5:7b` agent, plus three further passes (temperature, same-family, reference-shown) whose only purpose is to measure how much of the headline is the judge rather than the agent.

2.4 Baselines

B0 trivial: majority-class intent, one fixed routing decision (reported both ways), one canned "DM us" line. **B1 simple**: TF-IDF + logistic regression for intent, a hand-written keyword lexicon for routing, and a verbatim nearest-neighbour Spotify reply — no LLM anywhere. **S2** ablates the evidence from the draft; **S4** replaces the policy with an LLM router; **HUMAN** is the reply Spotify actually sent. B0 and B1 are fitted with 5-fold cross-validation *on the golden labels* while the LLM systems are zero-shot and never see one, so the baselines are deliberately allowed to cheat.

3. Results

Tables are generated by `src/report_tables.py` from `results/metrics.json`; the full set, including confidence intervals, per-intent breakdowns and handoff compliance, is in `results/tables.md`.

Table 1 - Headline comparison (n = 200 golden units)

system	intent macro-F1	intent acc	escalation recall	unsafe auto-handle	auto coverage	send-ready	TAR
B0 trivial (escalate all + canned reply)	0.031	18.5%	100.0%	0.0%	0.0%	10.0%	0.0%
B0 trivial (auto all + canned reply)	0.031	18.5%	0.0%	49.0%	100.0%	10.0%	2.0%
B1 simple (TF-IDF+LogReg / lexicon / NN reply)	0.295	41.0%	71.4%	14.0%	58.5%	50.5%	21.0%
S2 agent, no retrieval (ablation)	0.727	75.0%	93.3%	3.0%	48.0%	3.0%	3.0%
S4 agent, LLM router (ablation)	0.727	75.0%	55.6%	20.0%	73.0%	26.0%	13.0%
S3 agent (headline)	0.670	71.0%	86.7%	6.5%	50.0%	25.0%	10.5%
HUMAN @SpotifyCares reply actually sent	-	-	-	-	-	19.0%	-

Table 6 - Judge vs human on 48 blind-scored replies

dimension	human mean	judge mean	judge bias	within 1	QWK	Spearman
groundedness	4.12	4.40	+0.27	83.3%	0.26	0.28
resolution	3.33	3.35	+0.02	77.1%	0.21	0.32
tone	3.19	4.52	+1.33	54.2%	0.06	0.17
safety	4.06	4.69	+0.62	77.1%	0.28	0.40

Send-ready decision: human 29.2%, judge 37.5%, raw agreement 66.7%, kappa 0.2558. The judge waved through 10 replies a human would block and blocked 6 a human would send (false-pass rate 20.8% [0.12, 0.34]).

System ranking by mean rubric score - human: S3_agent > HUMAN_brand_reply > B1_simple > B0_trivial_escalate_all; judge: B1_simple > S3_agent > HUMAN_brand_reply > B0_trivial_escalate_all; identical: **False**.

3.1 The headline metric picks the wrong winner

B1 — TF-IDF nearest neighbour, returning a past Spotify reply verbatim — beats the agent on TAR: 21.0% vs 10.5% (27.5% vs 14.8% reweighted). The whole gap comes from the judge's send-ready rate (50.5% vs 25.0%), because TAR multiplies routing by reply quality and B1's routing is *worse* on every axis: escalation recall 71.4% against 86.7%, and an unsafe auto-handle rate of 14.0% — 0.14 against 0.065, so **2.2x the error that actually matters**.

So the composite I designed to have no trivial winner does have one, and it is the system that plagiarises. §3.3 shows why, and it is a flaw in my rubric rather than a virtue of B1.

The trivial baselines do their job: they expose bad metrics. Escalate-everything scores a perfect 100.0% escalation recall and a perfect 0.0% unsafe rate while automating nothing; auto-everything gets 100.0% coverage and lets through all 98 cases needing a human. Both have a TAR near zero, which is the one thing TAR does right.

The intent classifier is the least interesting result. Strict accuracy 71.0% (95% CI 0.644–0.768); lenient accuracy, which also accepts the alternative label I had already written down as defensible, is 83.0%. A logistic regression fitted on the gold labels manages macro-F1 0.295 against the agent's zero-shot 0.67 — but no intent number tells you whether the system is safe to deploy.

3.2 The ablations, matched on the same 100 units

Retrieval is what makes the replies usable at all. Triage never sees the evidence and the policy's retrieval clause is inert (§5.7), so S2 and S3 produce *identical* intents and routes by construction — the ablation isolates reply quality alone, and there the effect is enormous: send-ready 26.0% with evidence against 3.0% without. Ungrounded, the same model invents constantly: its unsupported-claim rate is 29.0% against 5.5% grounded.

Letting the LLM route is the clearest negative result here. On the same units, swapping the ~25-line deterministic policy for an LLM router:

	escalation recall	unsafe auto-handle	auto coverage	send-ready
S3, deterministic policy	93.3%	3.0%	48.0%	26.0%
S4, LLM router	55.6%	20.0%	73.0%	26.0%

It automates half again as much volume, writes equally good replies, and misses over 40% of the cases needing a human — a 6.7× increase in unsafe auto-handling. That is the empirical backing for putting policy in code (DECISIONS #4), and the answer I would give to "just let the model decide".

3.3 Reply quality: the judge prefers the plagiarist

The judge rubric means (Table 2 in [results/tables.md](#)) are the most important numbers here, and not for the reason I expected. B1 scores 4.79/5 on groundedness and 4.99/5 on safety — the highest of any system, including Spotify's own agents.

The mechanism is obvious in hindsight: **B1's reply is the evidence, so a groundedness dimension cannot fault it.** My anchors define groundedness as "are the claims supported", which says nothing about whether the reply is about the right subject; relevance lives in [resolution](#), where B1 still scores 3.465 — not low enough to fail the threshold. So the rubric systematically rewards copy-paste, and **had I selected a system using the judge alone I would have shipped the baseline that answered a request for one album with a paragraph about a different artist** (§4).

Two more things fall out of it. Spotify's own replies carry the *highest* unsupported-claim rate of any system (15.5% vs 5.5%) because real agents reference account records the judge cannot see — the judge penalises them for having information. And the judge's own [send_as_is](#) boolean says 87.5% where the threshold derived from its own numeric scores says 25.0%. That gap is why the harness defines send-ready in code instead of asking the model for a verdict.

4. Failure analysis: the five things that actually go wrong

Every example is a verbatim row from [results/failures.json](#).

F1. The retrieval baseline greets the wrong customer — and I mis-scored the humans for it

This starts as a finding about the system and ends as a finding about my evaluation. It is the most useful thing I learned.

What I saw while hand-scoring. Several replies greeted someone by a first name that appeared nowhere in the conversation — "Hey Jade", "Hey Jeremy", "Hi Allison", "Hey Sam", "Hey Tim". I deducted tone points each time.

Then I wrote a script to check my own annotations (`src/name_audit.py`). It extracts every greeting name and asks where it came from: this thread, the retrieved evidence only, or nowhere.

system	greeted by name	name in this thread	name only in retrieved evidence	name unverifiable
B1 simple (verbatim nearest neighbour)	5	0	5	0
HUMAN (Spotify's own reply)	6	1	0	5
S3 agent	0	–	–	–

(a) It is B1's defect, not the agent's. All of B1's named greetings use a name that provably belongs to a *different customer's* thread — the inevitable consequence of replaying a retrieved reply verbatim. The agent never greets anyone by name at all, which costs it warmth and removes this failure mode entirely. My first draft of this section blamed the agent; the script corrected me.

(b) I penalised Spotify's real agents for being right. Five of the six named greetings in Spotify's own replies use a name I could not find anywhere — because the dataset anonymises Twitter *handles* (`@115712` → `@customer`) but leaves first names in the reply body intact. The real agent could see the handle; I could not. Those were almost certainly the customer's actual name, and I marked them wrong.

(c) Correcting it erases the result I was most pleased with. Reverting only those five deductions (item-by-item in `src/name_audit.py` ; original blind scores kept as the primary record):

	agent, mean rubric	Spotify's own replies	agent send-ready	Spotify send-ready
as I scored it blind	4.067	3.85	8/15	4/15
after correcting my artefact	4.067	4.067	8/15	9/15

The agent-over-human margin goes from +0.217 to **0**, and on send-ready the humans move ahead. **The most flattering result in this project was an artefact of the dataset's anonymisation interacting with my annotation procedure**, and it survived until I wrote 60 lines of Python to interrogate my own labels. I have no reason to think it is the only one.

F2. The risk signals barely work — the intent is doing the escalating

gold escalation driver	n	model fired that signal	still escalated
<code>churn_threat</code>	8	100.0%	100.0%
<code>money_involved</code>	21	90.5%	95.2%
<code>security_or_privacy</code>	15	86.7%	100.0%
<code>account_specific</code>	28	60.7%	78.6%
<code>vague_no_detail</code>	9	33.3%	55.6%
<code>non_english</code>	13	7.7%	84.6%
<code>legal_or_regulatory</code>	2	0.0%	100.0%

Read the `non_english` row carefully. On the surface it looks fine — most non-English messages were escalated. But the language signal fired **once in thirteen**. I traced every one: **all 11 escalations happened because the model read the intent as `billing_charge`, `account_access` or `other_unclear`, which are never auto-handled anyway**. The two that leaked to auto-send were exactly the two whose intent was auto-eligible (Norwegian: "has Discover Weekly changed?", and an offer-eligibility question). The system has no language policy; it has a coincidence. A non-English feature request would be answered in English, publicly, with no human involved.

`legal_or_regulatory` never fires at all, and `account_specific` — the largest escalation driver in the golden set — is missed 39% of the time.

Hypothesis. Ten booleans in one call is too many. The model catches signals with loud lexical markers ("cancel", "charged", "hacked") and misses those requiring an inference about *what answering would require* (`account_specific`) or a judgement about the message as an artefact rather than its content (`non_english`). The fix is not a better prompt: language ID is solved and belongs in deterministic code before the LLM runs.

F3. Escalate-routed drafts try to solve the problem anyway

Of the 13 missed escalations, **7 produced a draft that attempted public resolution**; the other 6 said "DM us" anyway, making them label errors with little consequence. Routing and drafting are separate calls and nothing forces them to agree — hence handoff compliance as its own metric (Table 3b, `results/tables.md`).

Customer: "(One previous thread here `<url>` " (a fragment pointing elsewhere) **Draft:** "Thanks for reaching out, Jonny! We noticed your issue. Let's try restarting your device by holding the Home + Lock button for 10 seconds."

A name from nowhere, invented hardware steps for an unknown device, and a confident fix for a message containing no problem statement. **Fix:** when `route == escalate` , pick a handoff template and let the model fill only the acknowledgement.

F4. It invents plausible specifics when the evidence is generic

draft	the problem
"It sounds like Auto Skip might be on"	The evidence says <i>Autoplay</i> ; "Auto Skip" is not a Spotify feature.
"we'll re-tune Discover Weekly for you on Monday "	No such commitment exists in the evidence.
"Your monthly will pause automatically "	An invented billing policy.
"Suggest it in our Community: [link] "	An unfilled placeholder that would have been published.

Grounding also imports surface artefacts. Spotify's agents sign replies with their initials (/JX); the draft prompt explicitly forbids signing off. **21.0% of grounded drafts end in a fabricated agent signature anyway, against 1.0% for the same model with no evidence in the prompt** — the ablation makes the causal claim cheap. Retrieval teaches the model to invent a colleague's initials, and a written instruction does not stop it. Same mechanism as F1, different token.

Hypothesis. My **resolution** anchors punish "acknowledges and redirects" as a 3, so the prompt rewards concreteness while the evidence is often generic. Asked to be specific with nothing specific available, a 7B model produces specificity-shaped text. **My rubric creates the failure it then penalises:** **resolution** and **groundedness** are in tension and I set both.

F5. No abstention: vague messages get confident answers

Customer: "@SpotifyCares @customer" (two bare mentions, no content) **Draft:** "Hey there! We've got your back. You can remove songs from your library if needed." (routed auto)

By contrast, for "why is there limit for this? **<url>**" (referent inside an image) the agent asked a clarifying question and routed auto — exactly right. A clarifying question is the safest automatable reply there is, and my policy escalates **other_unclear** instead. That is the **clarify** tier in §6.

Intent confusions, and the baselines

Intent mismatches: 71.0% strict accuracy, so 58/200 disagree with my first-choice label — but 24 of those hit the alternative label I had already written down as defensible, leaving 34 genuine errors (83.0% lenient). Top confusions: **subscription_plan** → **billing_charge**, **playback_bug** → **complaint_no_ask**, **playback_bug** → **account_access**, **content_availability** → **feature_request**, **other_unclear** → **complaint_no_ask**. Two patterns: **lexical pull** (money words drag classification to billing regardless of what is asked — the same mechanism as the keyword baseline it should beat) and **reluctance to abstain** (**other_unclear** under-predicted, the classifier-side version of F5).

B1 fails differently and more dangerously — always fluent, sometimes about another subject:

Customer: "put [artist]'s *Welcome to Forever* album on Spotify please" **B1:** "Taylor Swift's 'Reputation' isn't available to stream just yet — stay tuned!"

A fluent wrong answer is worse than an obviously unhelpful one, because nobody reviews it.

5. What is misleading about my headline number

The section I care most about. Ten reasons, roughly by how much they should worry you.

5.1 The headline metric picks the wrong winner, and I left that in. TAR says the copy-paste baseline (21.0%) beats the agent (10.5%). I designed TAR so no trivial system could win it, and a trivial system won it. The reason is that TAR inherits the judge's send-ready rate, and §3.3 shows the judge is wrong about B1 in a specific, mechanical way. Two honest readings coexist: *as computed*, my system loses; *on every judge-independent axis* — escalation recall, unsafe auto-handle rate, intent macro-F1, unsupported-claim rate — it wins clearly. A composite metric does not launder a broken component, it propagates it.

5.2 The judge is an 8B model whose per-item decisions are barely better than noise. On 48 blind-scored replies it calls 37.5% send-ready where I called 29.2%; raw agreement 66.7%, kappa 0.256. Quadratic-weighted kappa per dimension is weak everywhere and near zero on tone (0.057, with a bias of 1.333 points). The false-pass rate is 20.8%: it waved through 10 replies I would have blocked. And it is unstable — rerunning the *same* judge at temperature 0.7 flips the send-ready verdict on 41.7% of replies. Read every send-ready figure as "according to an 8B judge that would disagree with itself two times in five".

5.3 The judge ranks the systems in the wrong order, and not subtly. By mean rubric score it ranks **B1 (copy-paste) > agent > Spotify's own replies**; on the same replies, hand-scored blind, I rank **agent ≥ Spotify's replies > B1**. It puts the plagiariser first and the professionals last. Any rubric with a groundedness axis should be run against a copy-paste control as a canary; mine would have failed that check before I quoted a number. Related: if a 7B model out-scores the humans it imitates, the rubric is rewarding *legible helpfulness* while Spotify's agents optimise for throughput and for moving to DM — which my **resolution** anchors punish as "acknowledges and redirects". **And self-preference is real and measurable:** swapping in a judge from the agent's own family widens the agent-over-human gap from 0.196 to 0.316.

5.4 Two of my own results did not survive auditing my own work. §4 F1: five deductions I made against Spotify's real replies were unjustified — I penalised them for greeting customers by names the dataset had hidden from me — and removing them collapses my hand-scored agent-over-human margin from +0.217 to 0. §4 F2: the escalation of non-English messages that looked like policy turned out to be a coincidence of intent classification. Both were found by writing code to interrogate my own artefacts, and both were flattering before I looked. I have no reason to think they were the last two.

5.5 The sample is enriched, so the on-sample number is not the inbox number. Security is 2.4% of the real pool and 8% of my sample; `safety_legal` 0.3% of the pool and 5% of my sample. Gold escalation rate is 49.0% on-sample, 38.1% reweighted. **Quote the reweighted column** — and note its own flaw: `general` carries weight 74 and `vague_short` 108, so a handful of units drive the population estimate, whose true variance is wider than the Wilson interval printed beside it. A stratum bootstrap would widen it further; I did not implement one.

5.6 Routing measures agreement with my policy, not with reality. Gold routes apply a risk appetite I chose, and the agent routes from signals defined in the same codebook I labelled from — so escalation recall partly measures whether the model and I read one document the same way. The external check (did Spotify actually DM?) I kept out of the labels deliberately.

5.7 One of my four routing conditions never fires. The policy escalates when best retrieval similarity is below 0.18; the minimum on the golden set is 0.199. A quarter of my "defence in depth" is decoration. The fix is calibrating on a held-out dev slice — not, as I was tempted, tuning on the golden set until the numbers improved.

5.8 $n=200$ means $\pm 7pp$ across ~ 24 reported numbers, with no multiplicity correction. Treat any gap narrower than $\sim 10pp$ as not demonstrated. The one comparison I would defend at this n is the router ablation, where the gap is $6.7\times$.

5.9 One annotator, and the consistency check is worthless. I re-labelled 40 units in a second pass and got 100% agreement (Table 7, `results/tables.md`). That is evidence of *memory* — same person, same day, items recognised. **Ignore that number.** Label uncertainty is unmeasured; the closest bound is the 23% `hard` rate, and the slice breakdown shows the agent's unsafe auto-handle rate is 17.4% on those units against 3.2% on the clear ones — a $5\times$ difference driven entirely by how hard I found the label.

5.10 Nothing here measures whether a customer was helped. No deflection, CSAT, re-contact or time-to-resolution — `resolution` is a model's guess about a reply whose consequences it never saw. `non_english` being a hard escalation signal conveniently removes the hardest generation cases from the quality metric (and §4 F2 shows that exclusion barely works anyway). The data is 2017, one brand, one channel, one language.

What I would actually put on a slide

On 200 held-out, deliberately risk-enriched messages from Spotify's public support inbox, the agent produced a send-ready, correctly-auto-routed draft for **10.5%** of them — **14.8%** reweighted to real inbox volume — which is **worse than a copy-paste baseline** on the same metric, for reasons I believe are a flaw in my judge rather than a virtue of the baseline. The numbers I would actually deploy against are judge-free: unsafe auto-handle rate **6.5%** (95% CI 0.038–0.108), escalation recall **86.7%**, at **50.0%** auto coverage. Before trusting any of it: one annotator, one brand, one language, a 2017 snapshot, and a judge with a 20.8% false-pass rate that disagrees with itself on 41.7% of replies.

6. What I would do with one more week

Ordered by expected value per day, not by how interesting it is.

Day 1 — fix the name and handoff bugs, then re-measure. Both are quantified in §4 and both get *guardrails* rather than better prompts, because a guardrail is unit testable: reject any first name in a draft that does not appear in the current thread, and fall back to a fixed holding template when an escalate-routed draft contains no handoff language.

Day 2 — a second annotator on 80 units. The biggest hole here. Hand over 80 stratified units including all 46 `hard` ones, compute real inter-annotator kappa, adjudicate into a v2 golden set. Every number in this report is conditional on labels only I have seen, and I cannot fix that alone.

Day 3 — calibrate the abstention rule that currently does nothing. Carve a 300-unit dev slice out of the pool (never the golden set), label it coarsely, and fit both the retrieval threshold (§5.7) and the confidence band, which currently trusts the model's self-reported `low/medium/high` with no evidence those words mean anything. Target a selective prediction curve so a support lead picks the operating point rather than accepting mine.

Day 4 — add the `clarify` tier. The biggest coverage win available: `other_unclear` and low-confidence cases go to a human today, but §4 F5 shows many need only "what's happening exactly, and on which device?". A clarifying question makes no commitments, so it adds almost no risk. Gate it hard — no clarify if any money/security/legal signal fires, and one clarify then escalate.

Day 5 — fix the judge, now the weakest component. §3.3 showed the rubric ranks a copy-paste baseline first, so: add a `relevance` dimension and make `groundedness` conditional on it; keep the copy-paste baseline in the comparison permanently as a canary; run it all past a stronger judge; bootstrap CIs over strata instead of Wilson; and add a prompt-injection slice — the golden set has none, and a public inbox is exactly where those arrive.

If a sixth day existed — measure something real. A shadow-mode harness: run the agent behind a live queue, send nothing, and have the human who answered each ticket mark whether they would have sent the draft. Two weeks of that beats everything in this repo, because the label comes from the person whose name goes on the reply.

Dhrumil Kadchha · Hiver SDE intern take-home · github.com/dkadchha2845/hiver-support-agent — `make setup` & `make repro` regenerates every number above from committed artefacts in **49s** on a fresh clone, byte-identically, without generating a token. Design reasoning: `DECISIONS.md`; labelling protocol: `data/golden/LABELLING.md`; Tables 3–5 and 7: `results/tables.md`.